



## Faculty of Technology and Engineering

### Computer Science and Engineering

#### Practical

|               |   |         |             |   |                               |
|---------------|---|---------|-------------|---|-------------------------------|
| Academic Year | : | 2025-26 | Semester    | : | 6                             |
| Course code   | : | CSE312  | Course name | : | Design of language processing |

#### Practical - 6

##### 1. Objective:

Implement a Recursive Descent Parser (RDP) to validate an input string against the given grammar.

$$S \rightarrow ( L ) \mid a$$

$$L \rightarrow S L'$$

$$L' \rightarrow , S L' \mid \epsilon$$

##### 2. Program Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
// Grammar:
// S -> ( L ) | a
// L -> S L'
// L' -> , S L' | epsilon
```

```
const char *input;
int pos;
int error;
```

```
void parse_S();
void parse_L();
void parse_Lprime();
```

```
char current() {
    return input[pos];
}
```

```
void consume(char expected) {
    if (input[pos] == expected) {
        pos++;
    } else {
```

```

    error = 1;
}
}

// S -> ( L ) | a
void parse_S() {
    if (error) return;
    if (current() == '(') {
        consume('(');
        parse_L();
        if (!error) consume(')');
    } else if (current() == 'a') {
        consume('a');
    } else {
        error = 1;
    }
}

// L -> S L'
void parse_L() {
    if (error) return;
    parse_S();
    parse_Lprime();
}

// L' -> , S L' | epsilon
void parse_Lprime() {
    if (error) return;
    if (current() == ',') {
        consume(',');
        parse_S();
        parse_Lprime();
    }
    // else epsilon - do nothing
}

int main() {
    char buf[1000];
    printf("Enter string: ");
    if (fgets(buf, sizeof(buf), stdin) == NULL) {
        printf("Invalid string\n");
        return 0;
    }
    // Remove newline
    int len = strlen(buf);
    if (len > 0 && buf[len-1] == '\n') buf[--len] = '\0';

    // Remove spaces
    char cleaned[1000];
    int ci = 0;
    for (int i = 0; i < len; i++) {

```

```
    if (buf[i] != ' ') cleaned[ci++] = buf[i];
}
cleaned[ci] = '\0';

input = cleaned;
pos = 0;
error = 0;

parse_S();

if (!error && input[pos] == '\0') {
    printf("Valid string\n");
} else {
    printf("Invalid string\n");
}

return 0;
}
```

### 3.Output:

```

● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] gcc -o 6 6.c
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: a
Valid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a,a),(a,a)
Invalid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a,(a,a),a)
Valid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] clear
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: a
Valid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a)
Valid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a,a)
Valid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a,(a,a),a)
Valid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a,a),(a,a)
Invalid string
○ debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] a)
zsh: parse error near `)'
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: a)
Invalid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a
Invalid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: a,a
Invalid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: a,
Invalid string
● debdootmanna@Debdoots-MacBook-Air ~/../../DLP [main] ./6
Enter string: (a,a),a
Invalid string

```